

Agent-Mediated Knowledge Graph Chat — High Level Design (HLD)

1 — Purpose & Scope

Purpose: Define architecture, modules, runtime interactions, and API boundaries for a web-based knowledge-graph chat where each node is an independent chat/context workspace and a Router Agent composes node contexts for a single global LLM instance.

Scope (MVP):

- Web frontend with interactive canvas (tldraw/Excalidraw) + tabbed chat per node.
- Backend services: Graph DB (Neo4j/ArangoDB), per-node vector stores (FAISS), Router Agent, Merge Agent, LLM adapter, version snapshots.
- Child nodes for corrections + merge workflow with semantic diffs.
- Single-user operation, MIT license, early open source MVP.

Out of scope for HLD: collaborative multiuser features, autoscaling across many tenants, advanced automatic conflict resolution (beyond sequential merge), full enterprise observability (HLD contains minimal ops guidance).

2 — Core Modules (Responsibilities)

Frontend

- **Canvas UI:** interactive node graph (create/drag/connect nodes).
- **Node Chat Panel:** tabbed chat per node, message list, compose box.
- **Diff Viewer:** semantic summary + textual diff, approve/reject merge.
- **Sidebar / MiniMap:** node list, filters (Fresh / Stale / Merge Pending).
- **Tech:** React, tldraw/Excalidraw, Monaco for diffs.

Backend API (FastAPI)

- Exposes REST/HTTP and WebSocket endpoints for realtime UI updates.
- Enforces API contracts and input validation.
- Coordinates Router, Agents, DB and vector store interactions.

Graph Store (Neo4j / ArangoDB)

- Stores nodes, edges, node metadata, version references.
- Query graph topology (dependencies, conflicts).
- Schema details reserved for LLD.

Vector Store per Node (FAISS / Weaviate)

- Stores embeddings of node messages/fragments.
- Provides top-K fragment retrieval for semantic search.

LLM Adapter

- Wraps calls to chosen LLM provider (OpenAI/Anthropic/local models).
- Handles rate limits, prompt templates, system prompts, function calls.
- Exposes unified interface for Router & Agents.

Router Agent (Orchestrator)

- Receives user query + active node id.
- Determines candidate nodes to include (graph traversal + relevance).
- Performs per-node semantic search on vector stores → top-K fragments.
- Composes context package (fragments + metadata header + commit hashes).
- Sends to LLM Adapter and returns raw LLM result to requesting node.

Merge Agent

- Accepts merge requests from child nodes.
- Computes textual diff between parent + child and produces semantic summary.
- Enforces merge policy (sequential in MVP).
- Writes snapshots and updates parent node if approved.
- Marks dependent nodes as stale.

Clarification Agent

- Scans for knowledge gaps/hallucination signals in LLM outputs.
- Auto-creates question/clarification nodes or prompts user to clarify.
- Logs hallucination events to node's hallucination_log.

Version Store

- Git-like snapshot storage for node states (GitPython or file store with metadata).
- Stores commit metadata, author, timestamp, diffs.

Cache & Session Manager

- In-memory LRU cache of hot node contexts and embeddings for performance.
- Session manager stores ephemeral session state for the single user.

Worker Queue

- Background queue (Redis + RQ or Celery) for expensive jobs: building embeddings, snapshot creation, staleness propagation, heavy diffing.

3 — Major Data Structures & Contracts (High-level JSON shapes)

Node (graph entity)

```
{
  "id": "uuid",
  "name": "string",
  "type": "main|component|child|continuation",
  "status": "fresh|stale|merge_pending|archived",
  "history": [{ "msg_id": "uuid", "role": "user|assistant|
system", "text": "...", "timestamp": "..."} ],
  "vector_index_id": "string",
  "version_log": [{ "commit_id": "sha", "timestamp": "..."} ],
  "hallucination_log": [{ "id": "uuid", "type": "MissingFact|WrongAssumption|
Speculation", "snippet": "...", "created_at": "..."} ],
  "metadata": { ... }
}
```

Fragment (retrieved from vector store)

```
{
  "node_id": "uuid",
  "fragment_id": "uuid",
  "text": "string",
  "embedding_meta": { "offset": 123, "commit_id": "sha", "created_at": "..."},
  "score": 0.987
}
```

Merge Request (child → parent)

```
{
  "child_id": "uuid",
  "parent_id": "uuid",
  "requested_by": "user_id",
  "diff_summary": "NL summary of changes",
  "text_diff": "unified diff or JSON patch",
  "status": "pending|approved|rejected",
  "created_at": "..."
}
```

Router Context Package

```
{
  "active_node": "Node_X",
  "selected_fragments": [ {"node_id": "...", "text": "...", "score": {...}}, {...} ],
  "metadata": { "nodes": [{"id": "...", "status": "fresh", "commit": "..."}] },
  "user_query": "..."
}
```

4 — Key Interaction Flows (Sequence diagrams)

A. Query → Router → LLM → Response

```
sequenceDiagram
    participant User
    participant Frontend
    participant Backend
    participant Router
    participant VectorStore
    participant GraphDB
    participant LLM
    User->>Frontend: send query in Node_A
    Frontend->>Backend: POST /router/query { node:A, query }
    Backend->>Router: routeQuery(node:A, query)
    Router->>GraphDB: fetch dependencies for Node_A
    Router->>VectorStore: semanticSearch(nodeX, query) (for each candidate)
    VectorStore-->>Router: topK fragments
    Router->>LLM: send context package + prompt
    LLM-->>Router: response
    Router-->>Backend: packaged response
    Backend-->>Frontend: return LLM result
    Frontend->>User: show response
```

B. Hallucination → Child Node Correction → Merge

```
sequenceDiagram
    participant User
    participant Frontend
    participant Backend
    participant MergeAgent
    participant VersionStore
    User->>Frontend: mark hallucinated text, click Edit
    Frontend->>Backend: POST /nodes/{parent}/child (create child node)
    Backend->>LLM: run correction session in child node
    LLM-->>Backend: corrected snippet
```

```

Frontend->>User: show child conversation
User->>Frontend: Request Merge
Frontend->>Backend: POST /merge/{child_id}
Backend->>MergeAgent: compute text diff + semantic summary
MergeAgent->>VersionStore: snapshot parent
MergeAgent->>Backend: diff + summary
Backend->>Frontend: show diff UI
User->>Frontend: Approve
Frontend->>Backend: POST /merge/{child_id}/approve
MergeAgent->>GraphDB: update parent node content, create commit reference
MergeAgent->>GraphDB: mark dependents stale
Backend->>Frontend: success

```

5 — Data Flow Diagram (conceptual)

User → Frontend (canvas, chats) → Backend API (router query, node CRUD, merge requests) → Backend orchestrates: GraphDB (topology), VectorStore (fragments), VersionStore (snapshots), LLM Adapter (model), Worker Queue (background jobs), Agents (Router/Merge/Clarify) → Backend → Frontend (responses, events, live updates).

6 — API Boundaries (public vs internal)

External API (Frontend ↔ Backend)

- `POST /router/query` — request composed answer for a node
- `GET /nodes` — list nodes + metadata
- `POST /nodes` — create node
- `GET /nodes/{id}` — fetch node preview & status
- `POST /nodes/{id}/messages` — append message to node
- `POST /nodes/{id}/child` — create child/correction node
- `POST /merge/{child_id}` — initiate merge
- `POST /merge/{child_id}/approve` — approve merge
- `GET /nodes/{id}/diff/{commit_a}/{commit_b}` — diff preview

Internal APIs / Services (Backend components)

- `RouterService.compose_context(node_id, query)` — returns context package
- `VectorService.search(node_id, query, top_k)` — internal retrieval
- `MergeService.compute_merge(child_id)` — internal diff & semantic summary
- `VersionService.create_snapshot(node_id)` — commit parent state
- `AgentBus.publish(event)` — background eventing for staleness propagation

Note: Keep LLM Adapter calls abstracted so model provider can be swapped without changing business logic.

7 — Non-functional Requirements & Decisions

Performance

- Target median LLM response time < 500ms for simple queries (model-dependent). For MVP, expect higher latencies.
- Router selects fragments to keep prompt length bounded (prefetch top-K fragments until token budget).

Scalability

- Per-node vector stores can be coalesced into a single index sharded by node for production.
- Worker queue for heavy tasks (embedding build, snapshot).

Consistency & Staleness

- Staleness model: update parent → mark dependents stale.
- Merge is manual/explicit; sequential merges prevent race conditions.

Fault Tolerance & Recovery

- Use worker queue retries & idempotent operations for snapshot and merge tasks.
- Store snapshots for rollbacks.

Security & Privacy

- Single-user mode for MVP simplifies access control.
- Encrypt storage at rest. Secure LLM keys in vault or env.
- Audit logs for merges & node edits.

Token Budgeting

- Node context token limit: 2,000,000 per node. Router must never overload the model: implement token accounting in Router.

8 — Key Assumptions & Constraints

- Single-user operation (no concurrent multi-user editing).
- Global LLM instance chosen; prompt engineering and token limits will affect retrieval strategy.
- Graph DB chosen: Neo4j or ArangoDB (the choice impacts query styles; LLD will include Cypher / AQL examples).
- Vector store initially FAISS local; later replaceable with managed services.

- Sequential merge policy is acceptable for MVP.
-

9 — Risks & Mitigations

- **Large prompts** → Risk: exceeding LLM token limits. Mitigation: Router selects top-K fragments and uses summarization to reduce size.
 - **Merge conflicts** → Risk: multiple child merges targeting same region. Mitigation: Sequential policy + conflict child node workflow for manual resolution.
 - **Staleness confusion** → Risk: user unaware dependents are stale. Mitigation: UI badges, prompts before main node aggregation.
 - **Hallucination detection false positives** → Risk: noisy metrics. Mitigation: store metadata, allow user to mark false positives and tune detection thresholds.
-

10 — Acceptance Criteria for HLD → LLD handoff

- Graph DB schema defined (node/edge fields, indices).
 - Router Agent interface + pseudocode available.
 - Merge Agent diff format + semantic summarizer contract.
 - API endpoints contract (request/response JSON schemas).
 - Token budget policy & example prompt templates.
-

11 — Next steps (deliverables I can produce immediately)

Pick one (or multiple) and I will produce it next:

- **LLD**: DB Schema + exact Cypher / AQL schema for Neo4j/ArangoDB (recommended next).
- **Router Agent pseudocode**: detailed function-level pseudocode including token budgeting & fragment ranking.
- ****M**